

React Interview Prep

Topic 17 — React Performance: Complete Guide

Measure First · DevTools Profiler · WDYR · 5 Problems & Fixes · Virtualization · State Colocation
· Context Splitting · Throttle & Debounce · Checklist · Q&A;

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 17

by Akshay Chavhan • React Interview Series

1. The Golden Rule — Measure First

Like a smart toy factory: only make toys when ordered, use shortcuts for repeats. React Performance = making React work smarter — only re-render what changed.

MEASURE → IDENTIFY → FIX

■ **Never optimise blindly. A fix without measurement can make things WORSE. Always open DevTools Profiler first.**

2. Tool 1 — React DevTools Profiler

Step	Action	What to look for
1	Install React DevTools Chrome extension	Profiler tab appears in DevTools
2	Open DevTools → Profiler tab	Flame graph view ready
3	Click Record ●	Recording starts
4	Interact with your app	Click, type, navigate
5	Click Stop ■	Flame graph renders
6 — Grey bars	Memoized — skipped ■	React.memo working
6 — Yellow bars	Rendered — took time ■■	Investigate this component
6 — Wide bars	More time spent rendering	Priority fix needed

3. Tool 2 — Why Did You Render (wdyr)

```
npm install @welldone-software/why-did-you-render

// wdyr.js – create this file, import BEFORE React in index.js
import React from 'react';
import whyDidYouRender from '@welldone-software/why-did-you-render';
if (process.env.NODE_ENV === 'development') {
  whyDidYouRender(React, { trackAllPureComponents: true });
}

// Mark specific component to track
MyComponent.whyDidYouRender = true;

// Console output – tells you EXACTLY why it re-rendered:
// 'MyComponent re-rendered because props.user changed'
// 'Previous: { name: Akshay } | Next: { name: Akshay }'
// Same value but NEW reference → useMemo fix needed
```

■ **WDYR shows you same-value-different-reference bugs instantly. This is the #1 cause of unnecessary re-renders in real apps.**

4. Problem 1 — Unnecessary Re-renders

Parent re-renders → all children re-render even if their props didn't change. Fix: the trio.

```
// The trio working together
const Child = React.memo(({ onClick, data }) => ( // memo on child
<div onClick={onClick}>{data.name}</div>
));
function Parent() {
const data = useMemo(() => ({ name: 'Akshay' }), []); // stable obj
const onClick = useCallback(() => console.log('hi'), []); // stable fn
return <Child onClick={onClick} data={data} />; // all props stable ■
}
```

■ *React.memo + useCallback + useMemo = the trio. All three needed — missing one defeats the others.*

5. Problem 2 — Expensive Computations on Every Render

■ Problem

```
// filter + sort runs on EVERY
render

// even when unrelated state
changes

function ProductList({ products,
query }) {
const filtered = products
.filter(p =>
p.name.includes(query))
.sort((a,b) => a.price - b.price);
// runs on every re-render ■
return filtered.map(p =>
<ProductCard key={p.id}
product={p}/>
);
}
```

■ Fix

```
// useMemo – only recompute when
deps change

function ProductList({ products,
query }) {
const filtered = useMemo(() =>
products
.filter(p =>
p.name.includes(query))
.sort((a,b) => a.price - b.price),
[products, query] // only reruns
here
);
return filtered.map(p =>
<ProductCard key={p.id}
product={p}/>
);
}
```

6. Problem 3 — Large Lists (Virtualization)

■ Problem

```
// 10,000 DOM nodes – browser  
chokes ■  
  
function UserList({ users }) {  
  return (  
    <div>  
      {users.map(u => (  
        <UserCard key={u.id} user={u} />  
      ))}  
      { /* 10,000 real DOM nodes */}  
      { /* browser freezes ■ */}  
    </div>  
  );  
}
```

■ Fix

```
// react-window – only visible  
rows ■  
  
import { FixedSizeList } from  
'react-window';  
  
function UserList({ users }) {  
  return (  
    <FixedSizeList  
      height={600}  
      itemCount={users.length}  
      itemSize={72}  
      width='100%'  
    >  
      {({ index, style }) => (  
        <div style={style}>  
          <UserCard user={users[index]} />  
        </div>  
      )}  
    </FixedSizeList>  
  );  
}  
  
// Renders ~8 rows – same speed  
always ■
```

■ *Virtualization = only render items visible in viewport. 10,000 items with react-window = same performance as 10 items.*

7. Problem 4 — Inline Objects & Arrays as Props

■ Problem

```
// New object every render
// defeats React.memo on Child ■
const Child = React.memo(({ style
}) =>
<div style={style} />
);

function Parent() {
  return (
    <Child
      style={{ color:'red', fontSize:16
    }}
    // ↑ NEW object every render
    // React.memo sees changed prop
    // Child re-renders ■
  />
  );
}
```

■ Fix

```
// Fix 1: static – move outside
component

const STYLE = { color:'red',
fontSize:16 };

function Parent() {

  return <Child style={STYLE} />; //
  stable ■
}

// Fix 2: dynamic – use useMemo
function Parent({ color }) {
  const style = useMemo(() => ({
    color, fontSize: 16
  })), [color]);
  return <Child style={style} />;
}
```

8. Problem 5 — Context Causing Too Many Re-renders

■ Problem

```
// One big context – any change
hits all ■

const AppContext =
  createContext(null);

function App() {
  const [user, setUser] =
    useState(null);

  const [theme, setTheme] =
    useState('light');

  const [cart, setCart] =
    useState([]);

  // ANY state change re-renders
  // ALL consumers ■

  return (
    <AppContext.Provider
      value={{user, theme, cart, ...}}>
    <Routes />
    </AppContext.Provider>
  );
}
```

■ Fix

```
// Split contexts – isolated
re-renders ■

const AuthContext =
  createContext(null);

const ThemeContext =
  createContext(null);

const CartContext =
  createContext(null);

function App() {
  return (
    <AuthContext.Provider
      value={{user, setUser}}>
    <ThemeContext.Provider
      value={{theme, setTheme}}>
    <CartContext.Provider
      value={{cart, setCart}}>
    <Routes />
    </CartContext.Provider>
    </ThemeContext.Provider>
    </AuthContext.Provider>
  );
}

// Cart change → only Cart
consumers re-render ■
```

9. State Colocation — Keep State Close to Use

■ Problem

```
// Search state in App – every  
keystroke  
  
// re-renders ENTIRE app ■  
  
function App() {  
  
  const [query, setQuery] =  
    useState('');  
  
  return (  
  
    <div>  
  
      <Header /> // re-renders ■  
  
      <SearchBar query={query}  
        onChange={setQuery} />  
  
      <Sidebar /> // re-renders ■  
  
      <Footer /> // re-renders ■  
  
    </div>  
  
  );  
}
```

■ Fix

```
// State colocated – only  
SearchFeature  
  
// re-renders on keystroke ■  
  
function App() {  
  
  return (  
  
    <div>  
  
      <Header /> // never touched ■  
  
      <SearchFeature /> // owns its  
        state  
  
      <Sidebar /> // never touched ■  
  
      <Footer /> // never touched ■  
  
    </div>  
  
  );  
}  
  
function SearchFeature() {  
  
  const [query, setQuery] =  
    useState('');  
  
  return (  
  
    <>  
  
      <SearchBar query={query}  
        onChange={setQuery} />  
  
      <SearchResults query={query} />  
  
    </>  
  
  );  
}
```

■ **State colocation = keep state as low in the tree as possible. Every level above where state lives will re-render when it changes.**

10. Throttle & Debounce — Limit Expensive Operations

■ Problem

```
// ■ Fires hundreds of times per second
useEffect(() => {
  window.addEventListener(
    'resize',
    handleResize // too frequent!
  );
}, []);
```

■ Fix

```
// ■ Throttle – max once per 100ms
import { throttle } from 'lodash';

useEffect(() => {
  const throttled =
    throttle(handleResize, 100);
  window.addEventListener('resize',
    throttled);

  return () =>
    window.removeEventListener('resize',
      throttled);
}, []);
```

Technique	How it works	Best for
Throttle	Fire at most once per time interval	scroll, resize, mousemove handlers
Debounce	Fire only after triggers stop for N ms	search inputs, form auto-save

```
const debouncedSearch = useCallback(
  debounce((query) => fetchResults(query), 300),
  []
); // fires 300ms after user stops typing ■
```

11. Key Web Performance Metrics

Metric	What it measures	Target
FCP — First Contentful Paint	When user first sees content	< 1.8s
LCP — Largest Contentful Paint	When main content loads	< 2.5s
TTI — Time to Interactive	When app responds to clicks	< 3.8s
CLS — Cumulative Layout Shift	Things jumping around	< 0.1
Re-render count	Too many = wasted CPU	Minimise with memo
Bundle size	Too large = slow first load	Analyse + split

12. Bundle Analysis & Quick Wins

```
# Analyse what's making your bundle large
npx vite-bundle-visualizer # for Vite
npx next-bundle-analyzer # for Next.js
```

Common culprit	Problem	Fix
moment.js	350kb — huge date library	Replace with date-fns (tree-shakeable)

lodash	Full import = large bundle	Import specific: import debounce from 'lodash/debounce'
All icons	Entire icon set imported	Import only what you use
Large pages	All code in one chunk	React.lazy + code splitting per route

13. Complete Performance Checklist

■ Measure first	React DevTools Profiler before any fix — no blind optimisation
■ React.memo	Wrap expensive child components that receive stable props
■ useCallback	Wrap handlers passed to React.memo children
■ useMemo	Wrap expensive computations + object/array props to children
■ Virtualize lists	Use react-window for lists with 100+ items
■ Split contexts	One context per concern — avoid one large AppContext
■ Colocate state	Keep state as low in the tree as possible
■ Lazy load routes	React.lazy + dynamic import per page
■ Lazy load heavy components	Charts, editors, video players
■ Debounce search	Wait for user to stop typing before fetching
■ Throttle scroll/resize	Limit handler frequency to once per 100ms
■ Static objects outside	Move { style, config } objects outside component
■ Analyse bundle	Remove/replace large unused libraries

14. Interview Q&A;

Q

Beginner

Q1. How do you identify performance problems in React?

→ Use React DevTools Profiler — record an interaction, check the flame graph for wide yellow bars meaning slow renders. The 'why-did-you-render' library logs exactly why each component re-rendered. Always measure first — never optimise blindly, as a fix without measurement can make things worse.

Q

Beginner

Q2. What is list virtualization and when should you use it?

→ Virtualization renders only items visible in the viewport instead of all items in the DOM. Libraries like react-window handle this. Use it when rendering lists with more than 100–200 items. 10,000 items with virtualization performs the same as 10 items without it.

Q

Intermediate

Q3. What is state colocation and why does it improve performance?

→ State colocation means keeping state as close as possible to the components that use it — not lifting it higher than necessary. If search state lives in App, every keystroke re-renders the entire tree. If it lives in SearchFeature, only that subtree re-renders. Unnecessary lifting = unnecessary re-renders.

Q

Intermediate

Q4. How do you fix a context causing too many re-renders?

→ Split the single context into multiple smaller contexts — one per concern. AuthContext for user data, ThemeContext for theme, CartContext for cart. Each consumer only re-renders when its specific context changes. Also wrap Provider values in useMemo so object references stay stable.

Q

Intermediate

Q5. What are the most common React performance mistakes?

→ Inline object and array props that create new references every render — defeating React.memo. State too high in the tree causing large subtrees to re-render. Not memoizing expensive computations. Rendering all items in a large list without virtualization. Single large context where any change re-renders all consumers.

Q

Expert

Q6. What is the difference between throttle and debounce?

→ Throttle limits how often a function fires — maximum once per time interval regardless of how many triggers. Use for scroll, resize, and mousemove handlers. Debounce delays the function until triggers stop — fires once after a pause. Use for search inputs where you wait for the user to stop typing.

Q

Expert

Q7. How do you reduce bundle size in a React app?

→ Use React.lazy with dynamic import for each route — loads pages on demand. Analyse the bundle with vite-bundle-visualizer or next-bundle-analyzer to find large culprits. Replace moment.js with date-fns, import lodash functions individually, import only the icons you use. Audit and remove unused dependencies.